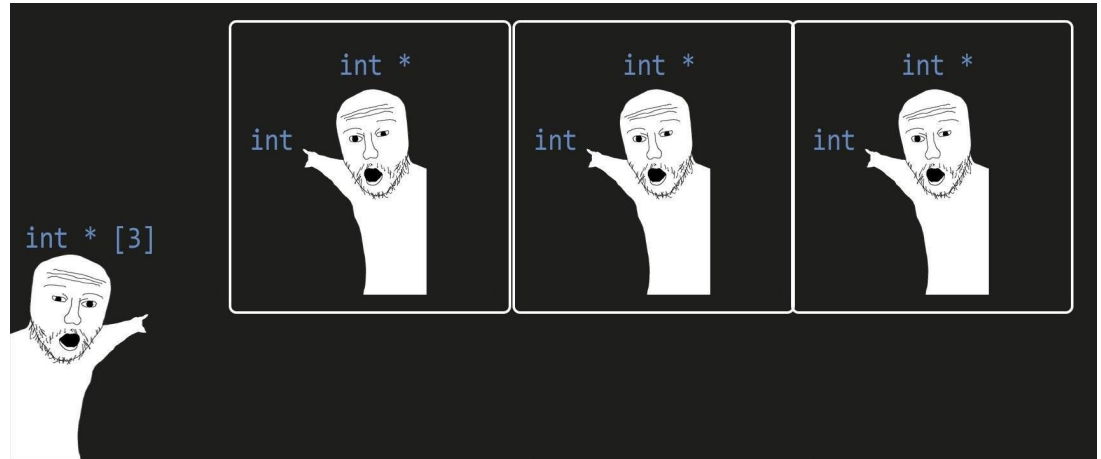
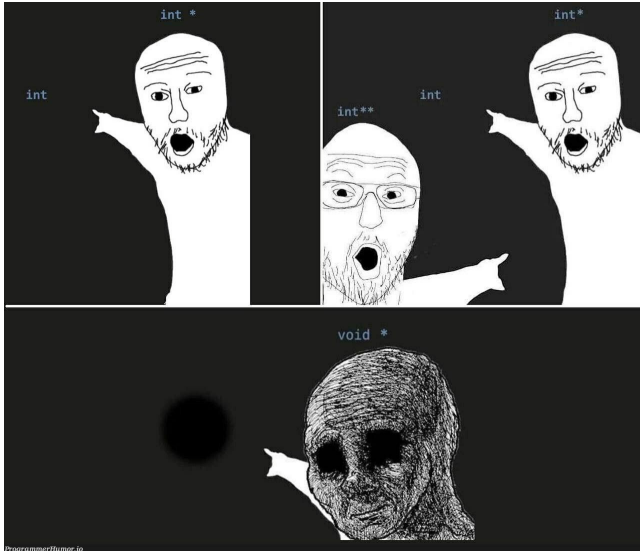


Memory Management - Pointers



**C++ developers after learning
pointers and references**



A program uses RAM, divided conceptually into regions.

Region	Purpose
Stack	Local variables, function calls
Heap	Dynamic memory (manual control)
Data / BSS	Global & static variables
Code (Text)	Program instructions

For DSA: stack + heap are the most important.

Stack vs Heap

Stack

- Automatic allocation & deallocation
- Limited size
- Function-scoped

```
void f() {  
    int x = 10;    // stack  
}
```

Heap

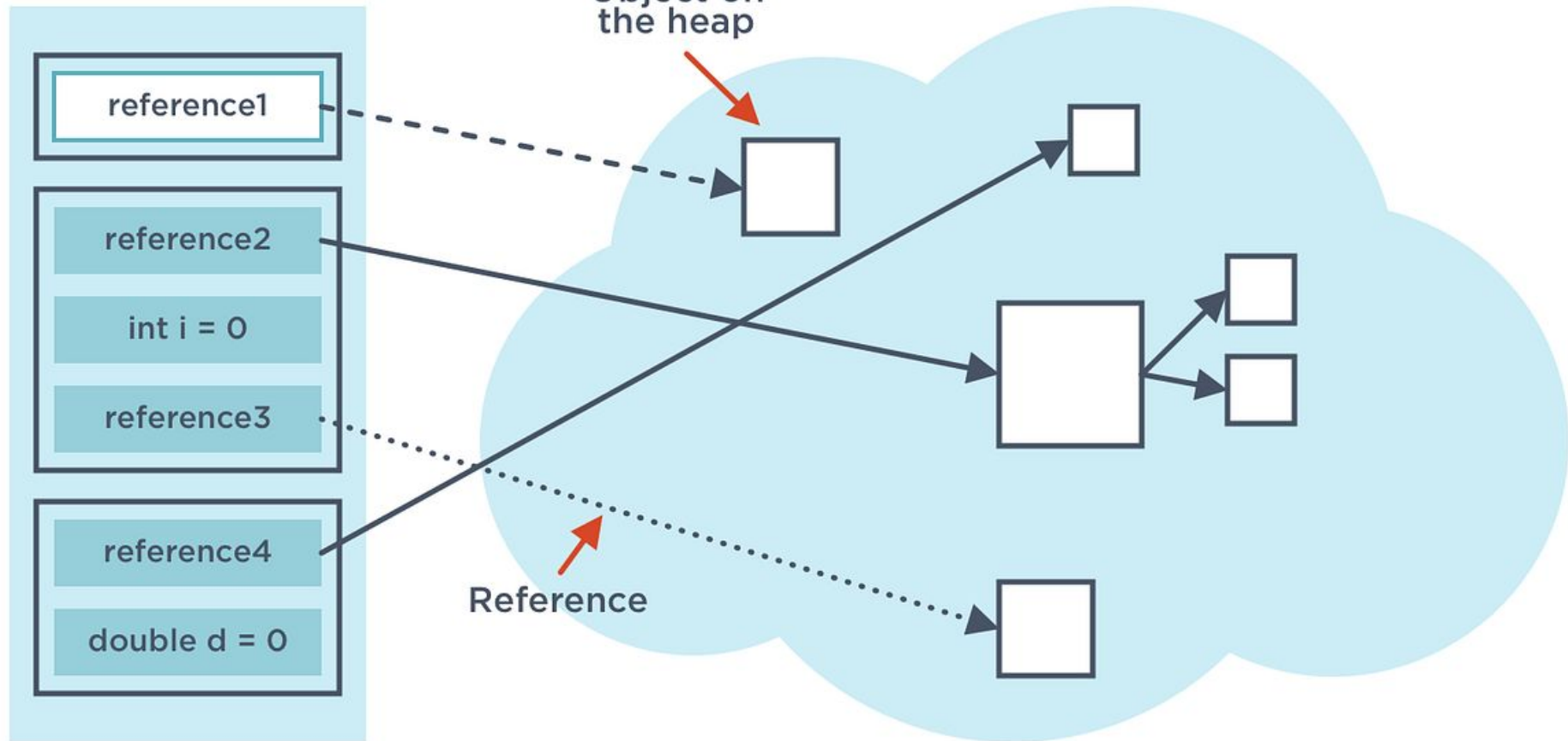
- Manual allocation
- Large
- Lifetime controlled by programmer

```
int* p = new int(10); // heap  
int* p = (int*)malloc(sizeof(int)); // heap
```

Stack is managed for you. Heap is managed by you.

Stack Memory

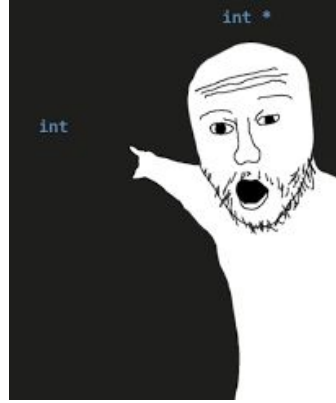
Heap Memory



What Is a Pointer?

A pointer is a variable that stores the address of another variable.

```
int x = 10;  
int* p = &x;
```



Address	Value
0x1000	10
0x1004	...
0x1008	0x1000
0x100c	...

Expression	Meaning
x	value
&x	Address of x
p	address of x
*p	value at address of x

Why Do We Use Pointers?

Pointers allow:

- Modifying variables across functions
- Dynamic memory allocation
- Efficient data structures
- Avoiding expensive copies

DSA truth

No pointers → no linked lists, trees, graphs.



Reference vs Dereference (Some english)

```
int x = 5;  
int* p = NULL;  
p = &x;  
*p = 9;
```

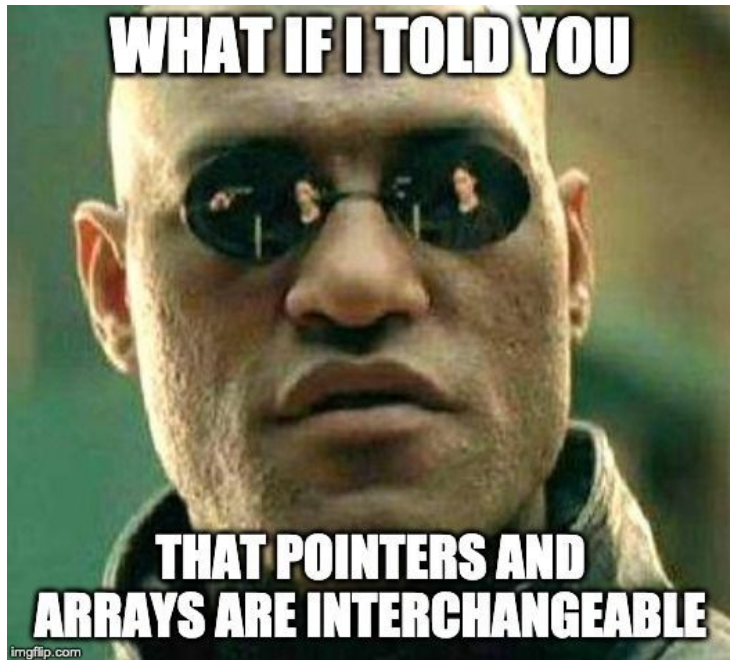
&x → reference (get address) : You apply this to the pointed guy

*p → dereference (get value) : You apply this to the pointer

pointer modifies original variable -> This is the foundation of pass-by-reference

Rule:

* means “go to that memory location”



Arrays



Pointers

Arrays Are Pointers (Contiguous memory)

```
int arr[3] = {10, 20, 30};
```

arr contains the address of the first item of the array

Key facts:

- $\text{arr} \rightarrow$ address of first element
- $\text{arr} == \&\text{arr}[0]$
- $\text{arr}[i] == *(\text{arr} + i)$

arr->

arr + 1->

arr + 2->

Address	Value
0x1000	10
0x1004	20
0x1008	30
0x100c	...

Dynamic Memory Allocation (Heap)

C Style

```
int* p = (int*)malloc(sizeof(int));  
*p = 10;  
free(p)
```

Arrays:

```
int* arr = (int*)malloc(sizeof(int) * 5);  
free(p)
```

C++ Style

```
int* p = new int; // or int* p = new int(10);  
*p = 10;  
delete p;
```

Arrays:

```
int* arr = new int[5];  
delete [] arr;
```

If you are not using your pointer immediately, make it NULL -> SegFault

You cannot dereference a pointer if it does not contain a valid address



Dangling pointer : A dangling pointer is a pointer that still holds the address of a memory location that is no longer valid



```
int* p = new int(10);
delete p;    // memory freed

// p is now a dangling pointer
cout << *p;  // undefined behavior
```

```
int* foo() {
    int x = 5;    // on stack
    return &x;    // XXX
}

int* p = foo(); // p is dangling
```



```
int* p = new int(10);
delete p;    // memory freed
p = NULL;
```

```
int* foo() {
    int* x = new int(5);
    return x;
}

int* p = foo(); // Yes my guy
```

Memory Lifecycle (Critical Concept)

1. **Allocate**
2. **Use**
3. **Free**

-  **Memory leak**
-  **Dangling pointer**
-  **Double free**

Modifying a Variable from a Function

```
void set5(int x) {  
    x = 5;  
}
```

```
// caller  
int x = 1;  
set5(x);
```

```
void set5(int* x) {  
    *x = 5;  
}
```

```
// caller  
int x = 1;  
set5(&x);
```

X in main

Copy of X

Pointer to X

Address	Value
0x1000	1
0x1004	1
0x1008	0x1000
0x100c	...

Returning an Array - The Trap

```
int* f() {  
    int arr[3] = {1,2,3};  
    return arr;  
}
```

```
// Heap Allocation  
int* f() {  
    int* arr = new int[3];  
    return arr;  
}
```

```
// caller frees  
int* a = f();  
delete[] a;
```

Pointers and Structs

```
struct Student {  
    int id;  
    float gpa;  
};
```

```
Student s;  
Student* p = &s;
```

Dereference

```
// option 1 : too verbose  
(*p).id = 10;  
  
// option 2 : Technolooooogia !!!  
p->id = 10;  
|  
// Then you can do something like  
Student* p = new Student;  
p->id = 1;  
p->gpa = 3.8;
```

Pointer to function

```
// some function
int square(int x) {
    return x * x;
}

// return_type (*pointer_name)(argument_types)
int (*f)(int) = square;

// use f as if it was a normal function
int y = f(5); // 25
```

Some resources I found useful

If you want a quick and high level
[youtube - Nesco Academy](#)



If you want to master, cook ++ pointers (3 hours video)
[youtube - freeCodeCamp](#)

